

ООП е парадигма, която се базира на обекти и операции над тях.

Четири основни принципа са:

1. Абстракция - работим с обектите чрез техния интерфейс без да познаваме реализацията им
2. Капсулация - вътрешното представяне на данните е скрито и промените по тях извън класа стават само чрез евидентни методи дефинирани за правят промени
3. Наследяване - нов клас се дефинира чрез разширяване на вече съществуващ
4. Полиморфизъм - единичен интерфейс с различно поведение спрямо типа на обекта

Абстрактен тип данни се представя чрез:

- логическо описание - какво представлява
- физическо представяне - как се съхранява
- операции - конструктори, селектори, мутатори,

Потребителят работи само с интерфейса, а след това представянето може да се промени без промени в кода на потребителя. В C++ абстрактен тип данни се реализира чрез класове

Класовете са потребителски дефинирани типове, а обектът е конкретен екземпляр от класа. Пример

```
class Dog {
```

```
private:
```

```
    int age;
```

```
    std::string name;
```

```
public:
```

```
    Dog(int age, std::string name) {
```

```
        this->age = age;
```

```
        this->name = name;
```

```
}
```

```
void makeSound() const {
```

```
    std::cout << "woof";
```

```
}
```

```
}
```

В горния пример имаме клас Dog с две публични функции, 1 конструктор с параметри и един метод. А в следния пример имаме обект от тип Dog:

```
Dog dog(5, "Кари");
```

Има 3 спецификатора за достъп:

- private - достъп само от публични функции и friend функции
- protected - позволява външен достъп но само за наследници
- public - неограничен външен достъп

По подразбиране в класовете имаме private достъп.

↓ това е как се защита с методи

Всяка публична функция (нестатична) ползва някакъв първи параметър `this`, което е указател към обекта от който е извикана. В примера с Dog го ползваме при конструктора, защото параметрите shadow-ват полетата ползваме са с едно и също име. Също така след списък с параметри на метод може да се сложи `const` (както е в Dog примера с `makeSound`) което прави `this` от тип `const Dog* const`, т.е. не може да се променя нито какъвто сочи указателят, нито обекта който сочи указателя. Също декларира преу компилатора, че този метод не мутира данните и по този начин компилаторът няма да даде да се извика метод, който не е деклариран като `const`.

↓ твърде подробно (трябва по-кратко) как работи може само ако има време

Основни видове конструктори

- конструктор с параметри `Dog(int age, std::string name)`
- конструктор с параметри по подразбиране `Dog(int age=2, std::string="Sharo")`
прави същото като конструктора с параметър но ако някой параметър липсва се използва default стойността
- конструктор по подразбиране - конструктор без параметри или без таки с подразбиращи се стойности. Вика се при `Dog dog;` или при `Dog* dogs = new Dog[100];`
- конструктор за копиране - `Dog(const Dog& other)`. Вика се при `Dog dog = dog2;` или `Dog dog(dog2);`

(Подава се const референция към обекта, който ще копираме)

- ако не дефинираме конструктор, то компилаторът генерира автоматично за нас конструктор по подразбиране, който за елементарни, които са обекти вика техния default конструктор, а примитивните типове оставя неинициализирани. (Също така се генерира и системен конструктор за копиране, който копира елемент по елемент. Прави плитко копие например ако имаме `char* data` ще направи копие на указателя и така два указателя ще сочат на едно място. Също така се дефинира оператор= който присвоява елемент по елемент (отново плитко копие) и деструктор, който не освобождава нищо, а само вика деструкторите на обектите). Ако се дефинира ксепвно и да е конструктор системен по подразбиране сигурно ще съществува.

При унивоксаване на обекта се вика автоматично деструктора, което за обекти на стека става след излизане от scope-а, а за динамични на heap след delete. За динамични масиви се вика `delete[]`

RAII

Ресурсът се придобива в конструктора и се освобождава в деструктора.
Така стандартът на ресурса се свързва с живота на обекта

```
class Array {
```

```
private:
```

```
    int* data;
```

```
    size_t size;
```

```
public:
```

```
    Array(size_t size = 0) : size(size), data(new int[size]) {} //придобиване
```

```
    Array(const Array& other) : size(0), data(nullptr) {
```

```
        *this = other;
```

```
    }
```

```
    Array& operator=(const Array& other) {
```

```
        if (this != &other) {
```

```
            delete[] data;
```

```
            data = new int[other.size];
```

```
            this->size = other.size;
```

```
            for (int i = 0; i < n; ++i) {
```

```
                this->data[i] = other.data[i];
```

```
            }
```

```
            return *this;
```

```
        }
```

```
    ~Array() {
```

```
        delete[] data; //освобождаване
```

```
    }
```

Голяма зетворка

Когато клас управлява динамичния памет той трябва да се погрижи тя да бъде освободена правилно, а системно генерираните конструктори, оператор= и деструктор не се справят с това. В тези случаи дефинирате

1. Конструктор по подразбиране

2. Конструктор за копиране - заделя нова памет и копира

3. оператор= - освобождава паметта, заделя нова и копира и задължително проверява `*this != &other`

4. Деструктор - освобождава памет

Методи

Методите се декларират и инициализират вътре в класа. Възможно е метод да бъде инициализиран извън класа, но задължително деклариран в класа

```
int Dog::getAge() {
```

```
    return age;
```

```
}
```

Всеки метод получава неявен параметър `this` от тип `T* const`.

Параметри може да се предават по:

- стойност (прави се копие)
- препратка - без копие, позволява промяна (`T&`)
- `const` препратка - без копие, не позволява промяна (`const T&`)
- по указател - когато искаме „личен обект“ `nullptr` (`T*`)

Връщане на резултат

- по стойност - прави копие
- по препратка или указател, но никога към локален обект, защото след това да съществува след изход от метода

Селекторите е добра практика да се декларират с `const` след списък с аргументи, защото не променят състоянието

Мутаторите променят състоянието

Наследяване

Производен клас разширява базов клас

```
class Person {  
    int age;  
    std::string name;  
  
public:  
    Person(int age=0, std::string name=""): name(name), age(age) {}  
    void print() const {  
        std::cout << "Name:" << name;  
    }  
}  
  
class Student: public Person {  
    int fn;  
  
public:  
    Student(std::string name, int age, int fn): Person(age, name), fn(fn) {}  
    void print() {  
        Person::print();  
        std::cout << "fn:" << fn;  
    }  
}
```

Производния клас получава всички полета и методи на базовия клас, но има достъп само до тези в категория public и protected.

Конструктора инициализира директно само своите данни, тези на базовия клас се инициализират на конструктор на базовия клас

Деструкторът на производния клас вика и деструктора на базовия клас

Вложени класове са класове дефинирани в тялото на други класове

```
class LinkedList {  
    struct Node {  
        int data;  
        Node* next;  
    };  
    Node* First;  
};
```

Вложеният клас се поддържа на контрола на достъпа. В случая горе с публичен избор класа, а ако беше в public частта щеше да може да се достъпи не само с `LinkedList::отпрез`.

Капсулацията служи за скриването на представянето (полетата са private) и за контролиран достъп до данните чрез селектори и мутатори.

По този начин:

- класът може да гарантира състояние (например denominator в Rational клас да е различен от 0)
- промените по данните на обекта стават само през мутаторите (по-лесно проследяване на грешки)

Статични полета и методи

Статичните полета принадлежат на класа, а не на конкретен обект.

Задават се еднократно и се споделят между всички обекти от дадения клас. Удобни са за например броячи на обекти, общи константи.

Деклариран се в класа, но се дефинират извън него.

Статичните методи са методи, които могат да се извикат без посредата от който конкретен обект. За да се достъпят се използва `име на клас :: име на метод`

